

NE 470/570 Monte Carlo Methods for Radiation Transport

Topic 2: Random Number Generator (RNG)

Xu Wu

xwu27@ncsu.edu

Associate Professor
Department of Nuclear Engineering
North Carolina State University

August 27, 2026

Outline

1. Random Number Generation Approaches
2. Pseudo Random Number Generators (PRNGs)
3. Randomness Testing
4. Examples

Outline

1. Random Number Generation Approaches
2. Pseudo Random Number Generators (PRNGs)
3. Randomness Testing
4. Examples

Random Number (RN) generation

- Random numbers (RNs) are an essential part of any Monte Carlo simulation.
- The quality of Monte Carlo simulation depends on the **quality (or randomness)** of the RNs used. A high degree of randomness is achieved if the RNs follow a **standard uniform distribution**.
- Therefore, we need to design approaches that yield sequences of numbers that:
 - 1 are **random**
 - 2 have a **long period** before repeating
 - 3 and **do not require significant resources** to generate

Random Number (RN) generation

- Random numbers (RNs) are an essential part of any Monte Carlo simulation.
- The quality of Monte Carlo simulation depends on the **quality (or randomness)** of the RNs used. A high degree of randomness is achieved if the RNs follow a **standard uniform distribution**.
- Therefore, we need to design approaches that yield sequences of numbers that:
 - 1 are **random**
 - 2 have a **long period** before repeating
 - 3 and **do not require significant resources** to generate
- Early implementation of RN generators on computers can be traced back to John von Neumann who used them for Monte Carlo simulation related to **the Manhattan project** (1941-1945).
- In this topic, we will:
 - 1 review different approaches for generation of RNs
 - 2 examine the behavior of the RN generators
 - 3 review several randomness tests, and elaborate on the impact of the parameters on the length of sequence and randomness of generated RNs.

Two common approaches for generating RNs: experimental and algorithmic

■ Experimental (look-up tables, online):

- 1 An experiment (physical process) is used to generate a sequence of RNs that are **saved in computer memory as tables**.
- 2 For example: (a) flip a coin or toss a die; (b) draw balls from an urn, i.e., lottery; (c) spin the marked disc; and, (d) measure the position of a dart from the center in a dart game.
- 3 These approaches are **quite slow** to be used during a Monte Carlo simulation.
 - Example: in an approach by Frigerio and Clark¹, the number of disintegrations of radioactive material over a given time interval, e.g., 20 ms, is counted, and if the number is odd, a 0-bit is recorded, otherwise a 1-bit is recorded. Then 31-bit numbers are formed. This process produces less than 6,000 RNs over an hour, which is quite slow. A tape containing 2.5×10^6 RNs is available at the ANL's code center.

■ Algorithmic (or deterministic):

- 1 An algorithm is used to generate RNs.
- 2 This approach, because of its **deterministic nature**, is commonly referred to as **pseudo random number generator (PRNG)** and its associated numbers are referred to as **pseudo random numbers (PRNs)**.

¹ N A Frigerio and N A Clark. Random number set for Monte Carlo computations. Technical report, 1975.

Metrics to evaluate a RN generation approach

■ Randomness:

- RNs should be truly random; i.e., they should follow a standard uniform distribution.
- In the experimental approach, randomness is achieved if the RNs follow a uniform distribution.
- In the algorithmic approach, the generated sequence has to satisfy several statistical tests.

■ Reproducibility:

- To test a simulation algorithm or to perform sensitivity/perturbation studies, it is necessary to be able to reproduce the RN sequence.

■ Length of the sequence of RNs:

- To perform a Monte Carlo simulation for realistic engineering problems, millions of RNs are needed; hence, the PRNG has to be able to produce a large number of RNs before repeating.

■ Computer memory:

- The generator should not consume too much computer memory.

■ Generation time:

- The amount of time (engineer/analyst) that it takes to generate a sequence of RNs should not be significant, e.g., days or even months.

■ Computer time:

- The amount of computer time needed to generate the RNs should be significantly shorter than the actual simulation.

Comparison of experimental and algorithmic RN generators

- In practice, the algorithmic approach is preferred mainly because its sequence is reproducible and it requires minimal effort (memory, time).
- In using any algorithmic generator, it is necessary to perform a series of randomness tests and to determine the period at which the generator repeats its sequence, i.e., loss of randomness.
- In the following, we will discuss different algorithmic PRNGs and randomness testing approaches.

Table 3.1 Comparison of experimental and algorithmic random number generators

Factor	Experimental		Algorithmic
	Table	online	
<i>Randomness</i>	good	good	to be tested
<i>Reproducibility</i>	yes	no	yes
Period ¹	limited	n/a	limited
<i>Computer memory</i>	large	small	small
<i>Generation time</i> ²	long	n/a	n/a
<i>Computer time</i>	short	long	short

¹ Maximum length of the sequence of RNs.

² Time needed to generate RNs.

Outline

1. Random Number Generation Approaches
2. Pseudo Random Number Generators (PRNGs)
3. Randomness Testing
4. Examples

Outline

2. Pseudo Random Number Generators (PRNGs)

2.1. Congruential Generators

2.2. Multiple Recursive Generator

Congruential generators

- A **congruential generator** is an integer generator proposed by D. H. Lehmer². It uses the following formulation:

$$x_{k+1} = (ax_k + b) \bmod M, \text{ for } b < M \quad (1)$$

where x_0 is called the **seed**, a (**multiplier**), b (**constant**) and M are given **integers**.

- For example, M as be chosen as the largest integer represented by a computer; e.g., on a binary machine with a 32-bit word length, the largest unsigned integer is $2^{32} - 1$ and the largest signed integer is $2^{31} - 1$.
- The modulus function determines the remainder of ($\alpha = ax_k + b$) divided by M . For example, $(35 \bmod 16 = 3)$. (Try $a = 5, b = 1, M = 16, x_0 = 1$)

²Derrick H Lehmer. Mathematical methods in large-scale computing units. Annu. Comput. Lab. Harvard Univ., 26:141-146, 1951.

- The term **congruential** comes from the fact that the ratios α/M and x_{k+1}/M have the same remainder, i.e., x_{k+1} is congruent to α modulo M .
- Equation (1) is referred to as a **linear congruential generator**, and, if $b = 0$, it is called a **multiplicative congruential generator**.
- A congruential generator will give a **random integer** x in the range of $[0, M - 1]$. To convert the random integers generated here into RNs in the range of $[0, 1]$, we use

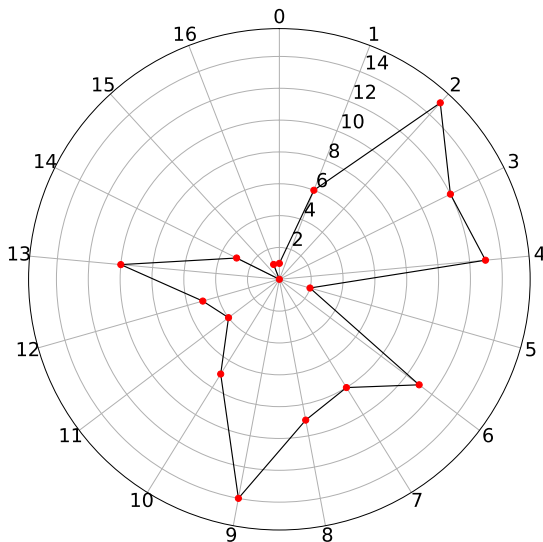
$$\eta = \frac{x}{M - 1} \quad (2)$$

Example of linear congruential generator

- Consider a linear congruential generator given by:

$$x_{k+1} = (5x_k + 1) \bmod 16 \quad (3)$$

- Considering a seed $x_0 = 1$, the sequence of RNs can be presented on a **random number cycle** (Note that the contours give the RN values, and the radial indices refer to order of the RN in the sequence.).
- The RN sequence has a period of 16; i.e., equivalent to the modulus, $M = 2^4$, and the largest number is $2^4 - 1$.
- Selection of any other number ($< M = 16$) as seed x_0 results in a cyclical shift of the above sequence.

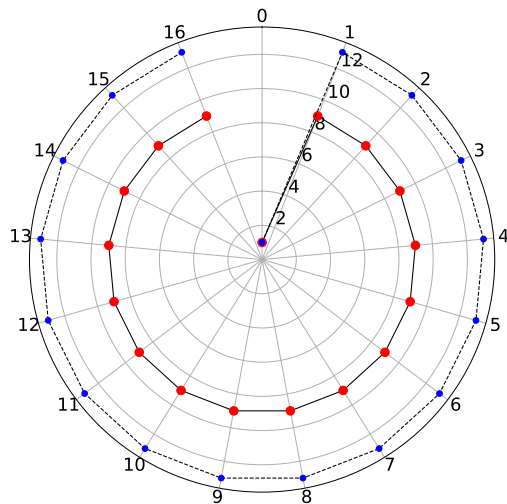
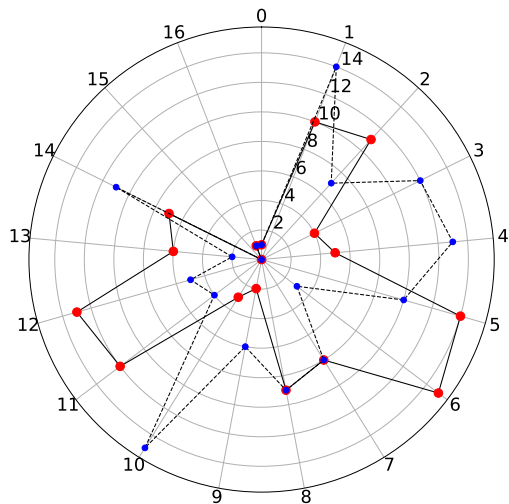


- Different multipliers a can have a significant effect on the period of the generator in Equation (3). If the multiplier (a) is equal to 5, 9, or 13, a full period of 16 is obtained, otherwise, partial periods of 2, 4, or 8 are achieved.

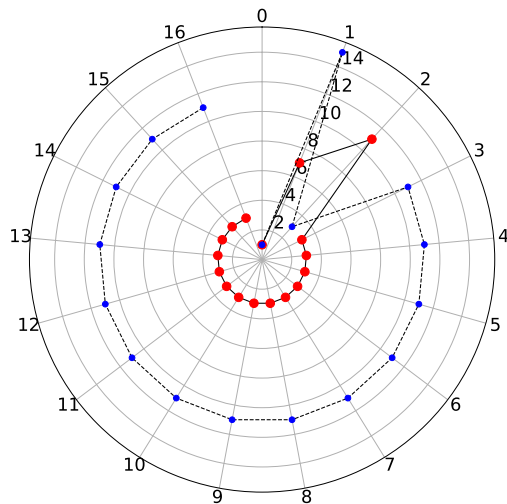
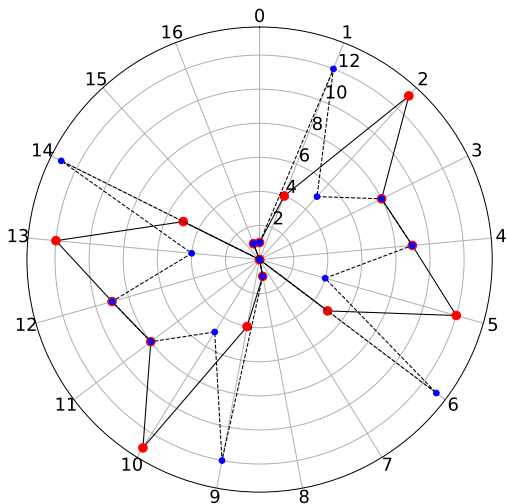
Table 3.2 Impact of *multiplier*(a) on the *period* of the PRNG (Eq. 3.2)

Parameter	Value												
Multiplier	3	4	5	6	7	8	9	10	11	12	13	14	15
Period	8	2	16	4	4	2	16	4	8	2	16	4	2

- Left figure: RN sequences for multipliers 9 (red) and 13 (blue), full period.
- Right figure: RN sequences for multipliers 8 (red) and 12 (blue), partial period.



- Left figure: RN sequences for multipliers 3 (red) and 11 (blue), partial period.
- Right figure: RN sequences for multipliers 6 (red) and 14 (blue), partial period.

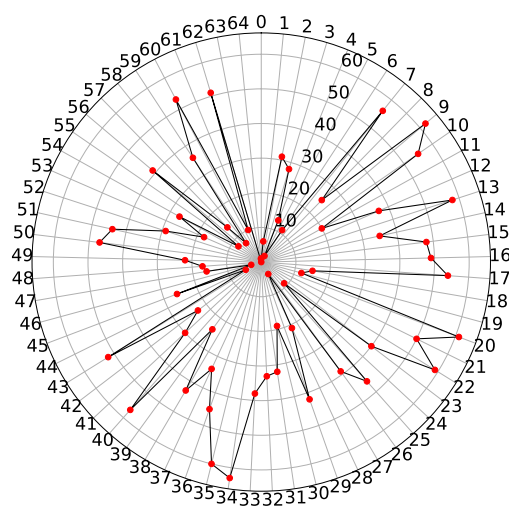
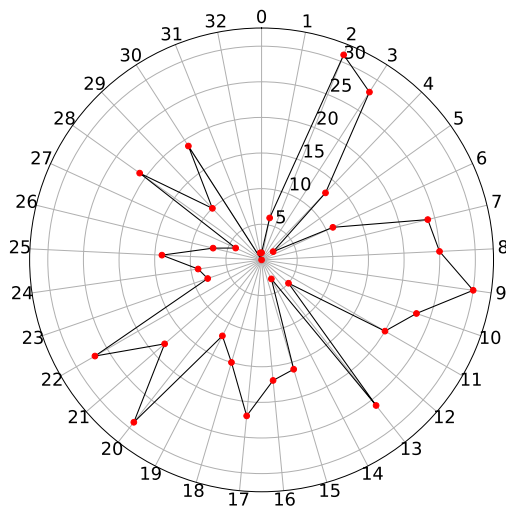


- Next, it is also important to examine the effect of the constant (b).
- For the case with multiplier $a = 5$, we determine the effect of constant (b) on the period of the generator.
- The table below compares the period of generator for different b values in a range of 2 to 8. It indicates that consistently an odd b results in a full period, while only a partial period is achieved for an even b .

Table 3.3 Impact of *constant*(b) on the *period* of the PRNG (Eq. 3.2)

Parameter	Value						
Constant	2	3	4	5	6	7	8
Period	8	16	2	16	8	16	4

- Considering the above observations, the generator $x_{k+1} = (5x_k + 1) \bmod 2^n$ should achieve a full period for any n .
- Left figure: RN sequences with $M = 2^5$. The period is $2^5 = 32$.
- Right figure: RN sequences with $M = 2^6$. The period is $2^6 = 64$.



- To achieve a **full period** for a **linear congruential generator**, we can use the corollary to the theorem by Hull and Dobell³ which sets the properties of different parameters including multiplier, constant and modulus.

Table 3.4 Properties of the parameters in a linear congruential generator for achieving a full period

Parameter	Value	Comment
multiplier (a)	$4N+1$	$N > 0$
constant (b)	odd number	
Modulus (M)	2^k	$k > 1$

³Thomas E Hull and Alan R Dobell. Random number generators. SIAM review, 4(3):230-254, 1962.

Example of multiplicative congruential generator

- For a **multiplicative congruential generator**, the constant parameter $b = 0$.
- For example

$$x_{k+1} = (ax_k) \bmod 16 \quad (4)$$

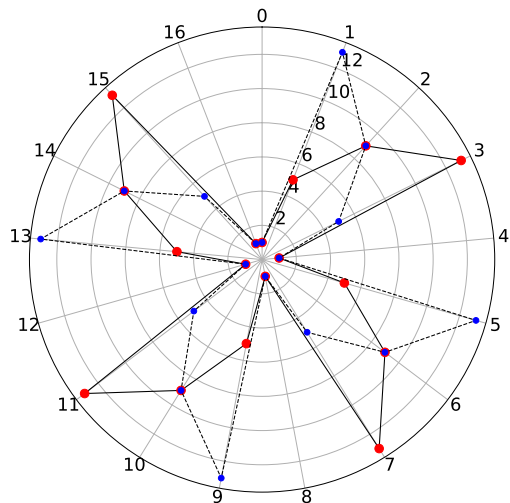
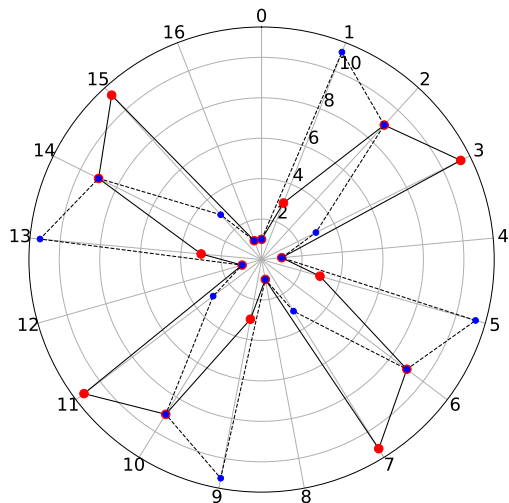
- The table below shows the period of RNs for different multipliers with a seed $x_0 = 1$.

Table 3.5 Impact of *multiplier* (a) on the *period* of the PRNG (Eq. 3.3)

Parameter	Value													
Multiplier	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Period	-	4	-	4	-	2	-	2	-	4	-	4	-	2

- This demonstrates the partial period of the generator as each case leads to polygons that are terminated by repeating the seed.
- These results indicate that for cases with $a = 8N + 3$ or $a = 8N + 5$ for $N \geq 0$, a multiplicative congruential generator yields a period of 2^{k-2} , where k corresponds to the exponent of modulus, e.g., $16 = 2^4$.

- Left figure: RN sequences generated by Equation (4) for two multipliers 3 (red) and 11 (blue).
- Right figure: RN sequences generated by Equation (4) for two multipliers 5 (red) and 13 (blue).



Outline

2. Pseudo Random Number Generators (PRNGs)

2.1. Congruential Generators

2.2. Multiple Recursive Generator

Multiple recursive generator

- **Multiple recursive generators** are a group of PRNGs that can be expressed by:

$$x_{k+1} = (a_0 x_k + a_1 x_{k-1} + \dots + a_j x_{k-j} + b) \bmod M$$

- One initiates the generator by selecting $(j + 1)$ RNs (possibly from a simpler generator). The length and the randomness of the generator depend on the values of a_j , b and M .
- A special case of this type of generator is the **Fibonacci generator**. A Fibonacci generator is a **floating point generator**. It is characterized by computing a new number by a combination (difference, sum, or product) of two preceding numbers. For example:

$$x_k = x_{k-17} - x_{k-5}$$

is a **Fibonacci generator of lags 17 and 5**.

- The generated sequence depends on the choice of the initial 17 numbers.
- Since the above Fibonacci formulation x_{k+1} depends upon 17 preceding values, its period (p) can be large, i.e., $p = (2^{17} - 1) \cdot 2^n$, where n is the number of bits in the fractional part of x_i , the **mantissa**. For example, for 32-bit floating point arithmetic, $n = 24$; hence, p is 2^{41} or 10^{12} .

- Because of the large expected period, a Fibonacci generator is a good choice for some large problems.
- For even larger problems on supercomputers, Fibonacci generators with lags 97 and 33 have been used.
- The table below shows some recommended Fibonacci generators. Note that the largest period that a **congruential generator** can have when using 32-bit integers is 2^{32} , or 4.3×10^9 , which is significantly **smaller** (at least by a factor of ~ 500) than Fibonacci generators in the table.

Table 3.7 A list of recommended Fibonacci generators

Generator	Expected period
$x_k = x_{k-17} - x_{k-5}$	$(2^{17} - 1) * 2^{24} = 2.2 * 10^{12}$
$x_k = x_{k-31} - x_{k-13}$	$(2^{31} - 1) * 2^{24} = 3.6 * 10^{16}$
$x_k = x_{k-97} - x_{k-33}$	$(2^{97} - 1) * 2^{24} = 2.7 * 10^{36}$

- A PRNG is acceptable if it passes a series of randomness tests, which look at various parameters for examining **independence** and **uniformity** of a RN sequence **based on their digits and as whole numbers**.

Outline

1. Random Number Generation Approaches
2. Pseudo Random Number Generators (PRNGs)
3. Randomness Testing
4. Examples

Outline

3. Randomness Testing

3.1. χ^2 test

3.2. Frequency Test

3.3. Serial Test

3.4. Gap Test

3.5. Poker Test

3.6. Moment Test

3.7. Serial Correlation Test

3.8. Serial Test via Plotting

The χ^2 distribution

- If a RV W follows a χ^2 distribution, its PDF is given by:

$$p(w) = \frac{w^{\frac{k}{2}-1} e^{-\frac{w}{2}}}{2^{\frac{k}{2}} \Gamma(\frac{k}{2})}, \quad 0 < w < \infty$$

where k is a positive integer, the **degree of freedom**.

- Γ is the **gamma function**.

$$\Gamma(z) = \int_0^{\infty} t^{z-1} e^{-t} dt$$

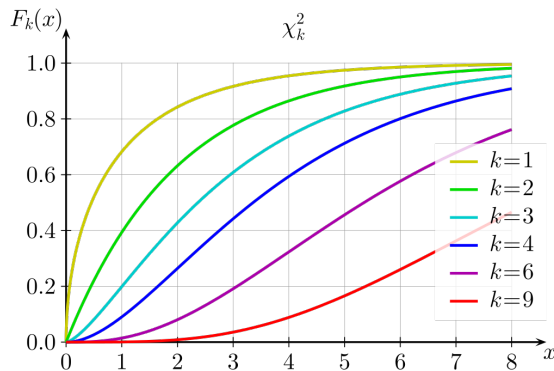
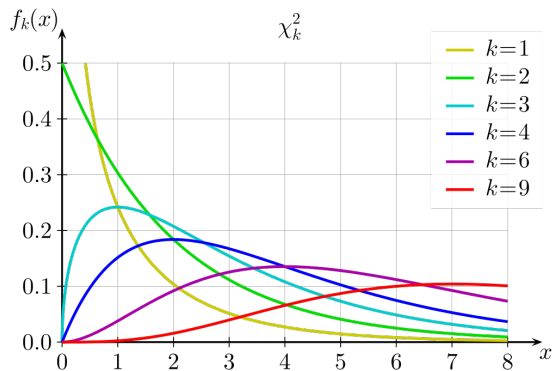
- The **χ^2 table** commonly gives **probabilities for $\chi^2 \geq \chi_0^2$** .
- To find $P(\chi^2 \leq \chi_0^2)$

$$P(\chi^2 < \chi_0^2) = 1 - P(\chi^2 \geq \chi_0^2)$$

Table 3.8 An example for χ^2 table ($P(\chi^2 \geq \chi_0^2)$)

Degrees of freedom	0.99	0.95	0.05	0.01	0.001
1	0	0.004	3.84	6.64	10.83
2	0.02	0.103	5.99	9.21	13.82
3	0.115	0.352	7.82	11.35	16.27
4	0.297	0.711	9.49	13.28	18.47
5	0.554	1.145	11.07	15.09	20.52
6	0.872	1.635	12.59	16.81	22.46
7	1.239	2.167	14.07	18.48	24.32
8	1.646	2.733	15.51	20.09	26.13
9	2.088	3.325	16.92	21.67	27.88
10	2.558	3.94	18.31	23.21	29.59

- If $Z_1, Z_2, \dots, Z_k$ are **independent, standard normal RVs**, then $W = \sum_{i=1}^k Z_i^2$ is distributed according to the χ^2 distribution with k degrees of freedom, $W \sim \chi^2(k)$ or $W \sim \chi_k^2$.
- The χ^2 distribution approaches a normal distribution as k increases, and the mean and the variance of the distribution are equal to k and $2k$, respectively.
- The figures below show the PDF and CDF of χ^2 distribution with different degrees of freedom⁴.



⁴https://en.wikipedia.org/wiki/Chi-squared_distribution

χ^2 Test

- The χ^2 test measures the deviation between the samples and the assumed probability distribution, i.e., hypothesis that the RNs follow a standard uniform distribution.
- The test statistic is given by:

$$\chi^2 = \sum_{i=1}^n \frac{(N_i - N p_i)^2}{N p_i} \quad (5)$$

where $\{p_1, p_2, \dots, p_n\}$ is a set of hypothetical probabilities associated with N events falling into n categories, with observed frequencies of $N_1, N_2, \dots, N_n$.

- For example, we can divide $(0, 1)$ into n bins. Under the hypothesis of a standard uniform distribution, each bin has a probability of $p_i = \frac{1}{n}$.
 - If there are N RNs in total, theoretically each bin should have $N p_i = N \cdot p_i$ samples.
 - The actual number of samples in the i^{th} bin is N_i .
- For large values of N , the random variable χ^2 approximately follows the χ^2 distribution with $(n - 1)$ degree of freedom.

Procedure for the use of χ^2 test

- The χ^2 test statistic values are obtained using Equation (5), and then these values are compared with the theoretical values using the χ^2 table.
- Commonly, we compare the estimated χ^2 values to theoretical χ_0^2 values that correspond to 5% probability. For example
 - 1 Divide the $(0, 1)$ space into 10 equally spaced bins. For a PRNG, we expect that every bin has an equal probability of 0.1, i.e., $p_i = 0.1$.
 - 2 After taking N random samples, we get N_i results in each bin. We can use these results to compute a χ^2 test statistic using Equation (5).
 - 3 It is expected that the estimated values of χ^2 with 9 degrees of freedom will be 16.92 at 5% probability.
 - 4 If the χ^2 value is larger than 16.92, the PRNG fails the test.
 - 5 **From the book:** *The estimated values of χ^2 with 9 degrees of freedom will be between the predicted values of 3.325 and 16.92 at 5% and 95% probabilities, respectively. If χ^2 is smaller than 3.325, the PRNG fails the test because its generated RNs are closer to each other than the hypothesis of standard uniform distribution. Conversely, if the χ^2 value is larger than 16.92, it means the RNG-generated numbers are too “variant” beyond the expected level. (This doesn't seem necessary, as we only need to look at one-side test.)*

An example for χ^2 test from the Fall 2021 midterm exam

- A 6-sided die is thrown 60 times. The number of times it lands with 1, 2, 3, 4, 5 and 6 face up is 7, 11, 19, 9, 8 and 6, respectively. Is the die biased, according to the χ^2 test at a significance level of 95% and 99%?

Degrees of Freedom	Chi-Square (χ^2) Distribution									
	Area to the Right of Critical Value									
	0.995	0.99	0.975	0.95	0.90	0.10	0.05	0.025	0.01	0.005
1	—	—	0.001	0.004	0.016	2.706	3.841	5.024	6.635	7.879
2	0.010	0.020	0.051	0.103	0.211	4.605	5.991	7.378	9.210	10.597
3	0.072	0.115	0.216	0.352	0.584	6.251	7.815	9.348	11.345	12.838
4	0.207	0.297	0.484	0.711	1.064	7.779	9.488	11.143	13.277	14.860
5	0.412	0.554	0.831	1.145	1.610	9.236	11.071	12.833	15.086	16.750
6	0.676	0.872	1.237	1.635	2.204	10.645	12.592	14.449	16.812	18.548
7	0.989	1.239	1.690	2.167	2.833	12.017	14.067	16.013	18.475	20.278
8	1.344	1.646	2.180	2.733	3.490	13.362	15.507	17.535	20.090	21.955
9	1.735	2.088	2.700	3.325	4.168	14.684	16.919	19.023	21.666	23.589
10	2.156	2.558	3.247	3.940	4.865	15.987	18.307	20.483	23.209	25.188
11	2.603	3.053	3.816	4.575	5.578	17.275	19.675	21.920	24.725	26.757
12	3.074	3.571	4.404	5.226	6.304	18.549	21.026	23.337	26.217	28.299
13	3.565	4.107	5.009	5.892	7.042	19.812	22.362	24.736	27.688	29.819
14	4.075	4.660	5.629	6.571	7.790	21.064	23.685	26.119	29.141	31.319
15	4.601	5.229	6.262	7.261	8.547	22.307	24.996	27.488	30.578	32.801

Solution

- The χ^2 test statistic is calculated by:

$$\chi^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i}$$

where O_i is the number of observations of type i . E_i is the theoretical/expected number of counts of type i according to the null hypothesis (in this case, uniform distribution). n is the total number of samples.

- The number of degrees of freedom is equal to the number of sample minus one, $\text{dof} = n - 1$.

i	O_i	E_i	$(O_i - E_i)$	$(O_i - E_i)^2$	$\frac{(O_i - E_i)^2}{E_i}$
1	7	10	-3	9	0.9
2	11	10	1	1	0.1
3	19	10	9	81	8.1
4	9	10	-1	1	0.1
5	8	10	-2	4	0.4
6	6	10	-4	16	1.6

- Based on the calculation in the table, the χ^2 test statistic is

$$\chi^2 = 0.9 + 0.1 + 8.1 + 0.1 + 0.4 + 1.6 = 11.2$$

- The $\text{dof} = 6 - 1 = 5$.
- The critical value at a significance level of 95% is **11.071**. The χ^2 test statistic **exceeds this critical value**. Therefore, we **reject the null hypothesis** and conclude that the die is **biased at 95% significance level**.
- The critical value at a significance level of 99% is **15.086**. The χ^2 test statistic **does not exceed this critical value**. Therefore, we **cannot reject the null hypothesis**. We conclude that there is **insufficient evidence to show that the die is biased at 99% significance level**.

Outline

3. Randomness Testing

3.1. χ^2 test

3.2. Frequency Test

3.3. Serial Test

3.4. Gap Test

3.5. Poker Test

3.6. Moment Test

3.7. Serial Correlation Test

3.8. Serial Test via Plotting

Frequency test

- In this test, one counts the number of occurrences of every digit (0 through 9) by parsing digits in every RN generated by a PRNG.
- Knowing the expected occurrence probability of $1/10$, one can calculate the χ^2 values to determine the degree of randomness of the PRNG.

Outline

3. Randomness Testing

3.1. χ^2 test

3.2. Frequency Test

3.3. Serial Test

3.4. Gap Test

3.5. Poker Test

3.6. Moment Test

3.7. Serial Correlation Test

3.8. Serial Test via Plotting

Serial test

- The serial test is an extension of the frequency test to **pairs of digits**.
- For any selected digit, one counts the occurrence of every other digit following the selected digit.
- Again, the expected chance of occurrence is $1/10$, and χ^2 values provide information on the degree of randomness of the PRNG.

Outline

3. Randomness Testing

3.1. χ^2 test

3.2. Frequency Test

3.3. Serial Test

3.4. Gap Test

3.5. Poker Test

3.6. Moment Test

3.7. Serial Correlation Test

3.8. Serial Test via Plotting

Gap test

- In the gap test, a particular digit, e.g., 0, is selected and the frequency of occurrence of nonzero digits between successive 0's is determined.
- For a single gap, the expected frequency is 9/100.
- Again, the χ^2 values are calculated to measure the quality of PRNG.

Outline

3. Randomness Testing

3.1. χ^2 test

3.2. Frequency Test

3.3. Serial Test

3.4. Gap Test

3.5. Poker Test

3.6. Moment Test

3.7. Serial Correlation Test

3.8. Serial Test via Plotting

Poker test

- In this test, the digits are partitioned into groups of five and the relative frequencies of “five of a kind,” “four of a kind,” etc., are determined.
- Again, χ^2 values are calculated to measure the quality of the PRNG.

Outline

3. Randomness Testing

3.1. χ^2 test

3.2. Frequency Test

3.3. Serial Test

3.4. Gap Test

3.5. Poker Test

3.6. Moment Test

3.7. Serial Correlation Test

3.8. Serial Test via Plotting

Moment test

- The k^{th} moment of a RV Y is defined as:

$$\mu_k = \int_a^b y^k p(y) dy \quad (6)$$

- Since a RN sequence should be distributed uniformly within the unit interval $[0, 1]$, for a standard uniform RV X , its k^{th} moment is:

$$\int_0^1 x^k dx = \frac{1}{k+1} \quad (7)$$

- Consequently, the randomness of a sequence of RNs, η 's can be tested by examining the following equality:

$$\frac{1}{N} \sum_{i=1}^N \eta_i^k \approx \frac{1}{k+1} \quad (8)$$

- Note that for $k = 1$, the above equation reduces to a simple average of RNs that is equal to 0.5.

Outline

3. Randomness Testing

3.1. χ^2 test

3.2. Frequency Test

3.3. Serial Test

3.4. Gap Test

3.5. Poker Test

3.6. Moment Test

3.7. Serial Correlation Test

3.8. Serial Test via Plotting

Serial correlation test

- The **serial correlation coefficient** for a sequence of RNs x_i of length N and lag j is given by:

$$\rho_{N,j} = \frac{\frac{1}{N} \sum_{i=1}^N x_i x_{i+j} - (\bar{x})^2}{\frac{1}{N-1} \sum_{i=1}^N x_i^2 - (\bar{x})^2} \quad (9)$$

where $\bar{x}$ is the mean value defined as

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad (10)$$

- If x_i and x_{i+j} are independent and N is large, then the correlation coefficient ($\rho_{N,j}$) should follow a normal distribution with mean $-\frac{1}{N}$, and standard deviation $\frac{1}{\sqrt{N}}$.

Outline

3. Randomness Testing

3.1. χ^2 test

3.2. Frequency Test

3.3. Serial Test

3.4. Gap Test

3.5. Poker Test

3.6. Moment Test

3.7. Serial Correlation Test

3.8. Serial Test via Plotting

Serial test via plotting

- As RNs are generated, they are combined, e.g., paired or 2-tuple, (x_1, x_2) , $(x_3, x_4), \dots$, and each pair (or combination) is plotted as a **point in a unit square**.
- If there is **any obvious pattern**, e.g., **striped**, then it can be concluded that the numbers are **serially correlated**.

Outline

1. Random Number Generation Approaches
2. Pseudo Random Number Generators (PRNGs)
3. Randomness Testing
- 4. Examples**

- Given a linear congruential generator

$$x_{k+1} = (ax_k + b) \bmod M$$

- Our objective is to examine the effect of **seed** (x_0), **multiplier** (a), and **constant** (b) on the randomness of the generator.
- We compare different sets of parameters by determining the period of the sequence, average, i.e., first moment of the RNs, and plotting of positions within a 3-D domain.

Outline

4. Examples

4.1. Evaluation of PRNG based on period and average

4.2. Serial Test via Plotting

Evaluation of PRNG based on period and average

- First, we examine the effect of the seed (x_0) in a multiplicative congruential generator with an odd multiplier (65539), shown in Table 3.9.
 - The table indicates that the seed has a major impact on the period of the sequence.
 - It confirms that, for an odd multiplier with an odd seed, a 25% period is achieved.
 - In Case 3, with an even but not power-2 seed, a reduced period of 12.5% is observed, while for Cases 4 and 5, with power-2 seeds, poor periods are expected.
 - Although, in all of these cases, the average is correct.

Table 3.9 Impact of *seed* on a multiplicative congruential generator

Parameter	case	<i>seed</i>	<i>a</i>	<i>b</i>	<i>M</i>	<i>period</i>	$\frac{\text{period}}{M}$	average
seed	1	1	65539	0	2^{24}	4194303	25%	0.500
	2	69069	65539	0	2^{24}	4194303	25%	0.500
	3	16806	65539	0	2^{24}	2097152	12.5%	0.500
	4	1024	65539	0	2^{24}	4096	0.024%	0.500
	5	4096	65539	0	2^{24}	1024	0.006%	0.500

- For a better understanding of **the impact of the seed x_0** , in Table 3.10, we determine the distribution of RNs within the range of $[0, 1]$.
- Table 3.10 demonstrates that Cases 4 and 5 not only have a very short period, but also result in a biased distribution of RNs.

Table 3.10 Impact of *multiplier* on a multiplicative congruential generator

Interval	Case 1	Case 2	Case 3	Case 4	Case 5
0.0-0.1	10%	10%	10%	10.01%	10.06%
0.1-0.2	10%	10%	10%	10.01%	10.06%
0.2-0.3	10%	10%	10%	10.01%	9.96%
0.3-0.4	10%	10%	10%	9.99%	9.96%
0.4-0.5	10%	10%	10%	9.99%	9.96%
0.5-0.6	10%	10%	10%	10.01%	10.06%
0.6-0.7	10%	10%	10%	10.01%	10.06%
0.7-0.8	10%	10%	10%	10.01%	9.96%
0.8-0.9	10%	10%	10%	9.99%	9.96%
0.9-1.0	10%	10%	10%	9.99%	9.96%

- Second, we examine the effect of the multiplier (a) for a fixed odd seed of $x_0 = 1$.
- Table 3.11 compares the periods and averages for odd and even multipliers.
- Table 3.11 indicates that the multiplier has a significant impact on the performance of a PRNG.
- Basically, if the multiplier is a power of 2 (similar to the modulus) (Cases 7 and 9), the generator fails; otherwise, for a large odd multiplier, the generator behaves as expected.

Table 3.11 Impact of *multiplier* on a multiplicative congruential generator

Parameter	case	seed	a	b	M	period	$\frac{\text{period}}{M}$	average
multiplier	6	1	69069	0	2^{24}	4194303	25%	0.500
	7	1	1024	0	2^{24}	3	0	0.021
	8	1	1812433253	0	2^{24}	4194303	25%	0.500
	9	1	4096	0	2^{24}	2	0	0.0
	10	1	1	0	2^{24}	1	0	0.0

- Third, we examine the effect of constant (b), for a fixed seed and multiplier, as given in Table 3.12.
- It is evident that the impact of constant (b) is not as significant as either multiplier or seed.
- However, the results indicate that the constant can significantly impact the period for a given choice of multiplier and seed.

Table 3.12 Impact of *constant* on a linear congruential generator

Parameter	case	seed	a	b	M	period	$\frac{\text{period}}{M}$	average
constant	11	1	65539	1	2^{24}	4194304	25%	0.500
	12	1	65539	1024	2^{24}	4194304	25%	0.500
	13	1	65539	4096	2^{24}	4194303	25%	0.500
	14	1	65539	65539	2^{24}	8388608	50%	0.500
	15	1	65539	69069	2^{24}	8388608	50%	0.500
	16	1	65539	$2^{24} - 1$	2^{24}	8388608	50%	0.500

- To explore this further, in Table 3.13, we examine small changes in the multiplier from 65539 to 65549 with an increment of 2, i.e., only odd multipliers, and fix seeds and constants of 1.
- Table 3.13 indicates that the generator has truly significant dependency on the multiplier because Cases 18, 20, and 22 show full periods as compared to the rest with partial periods.

Table 3.13 Impact of small changes of *multiplier* on a linear congruential generator

Parameter	case	seed	a	b	M	period	$\frac{\text{period}}{M}$	average
multiplier	17	1	65539	1	2^{24}	4194304	25%	0.500
	18	1	65541	1	2^{24}	16777216	100%	0.500
	19	1	65543	1	2^{24}	4194303	25%	0.500
	20	1	65545	1	2^{24}	16777216	100%	0.500
	21	1	65547	1	2^{24}	8388608	50%	0.500
	22	1	65549	1	2^{24}	16777216	100%	0.500

- The cases with full periods have a similar characteristic - their multipliers can be represented by the following formulations:
 - Case 18: multiplier = $65541 = 4 \times 16385 + 1$
 - Case 20: multiplier = $65545 = 4 \times 16386 + 1$
 - Case 22: multiplier = $65549 = 4 \times 16387 + 1$
- The above observation for Case 18, Case 20 and Case 22 provides the necessary condition for achieving a full period, as these cases' properties are similar those stated in the corollary to the Hull and Dobell's theory (Table 3.4).

Table 3.4 Properties of the parameters in a linear congruential generator for achieving a full period

Parameter	Value	Comment
multiplier (a)	$4N+1$	$N > 0$
contact (b)	odd number	
Modulus (M)	2^k	$k > 1$

Outline

4. Examples

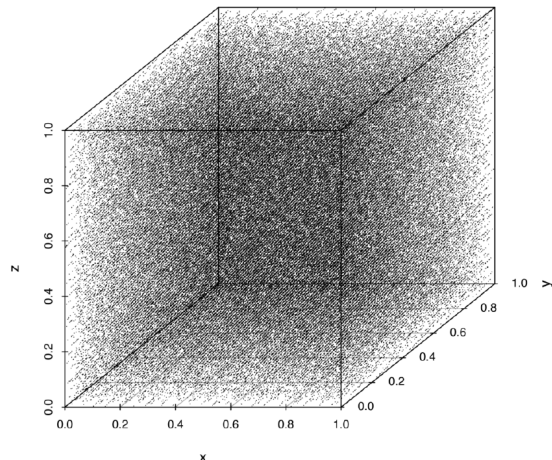
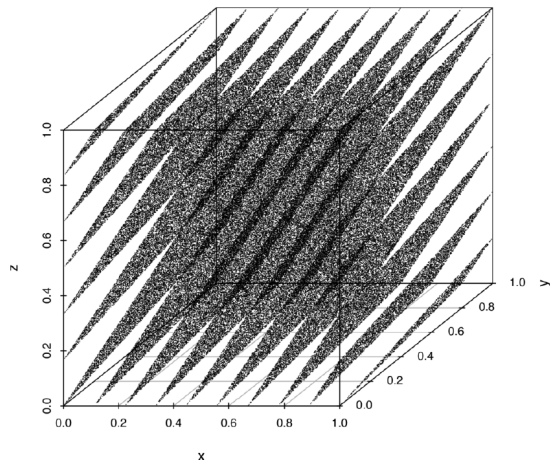
4.1. Evaluation of PRNG based on period and average

4.2. Serial Test via Plotting

Serial test via plotting

- Here, we examine two linear generators with different multipliers. Our aim is to demonstrate the potential benefit of the plotting approach, which can exhibit correlations among the RNs.
- First, we examine, two generator cases as follows:
 - 1 Case 17: $a = 65539$, seed $x_0 = 1$, $b = 1$, $\text{mod} = 2^{24}$
 - 2 Case 18: $a = 65541$, seed $x_0 = 1$, $b = 1$, $\text{mod} = 2^{24}$
- As indicated, Case 17 led to a period of 25% of the modulus and Case 18 resulted in a period equal to the modulus.
- To examine the randomness via plotting, we generated 3-tuple (x, y, z) positions using every three consecutive RNs and then marked the position in a 3-D domain.

- Left (Cases 17), right (Cases 18)
- These figures indicate that both generators lead to correlated (especially, Case 17) sets of RNs, even though Case 18 achieves a full period.



- Finally, we examine Case 23 with a multiplier of $a = 16333$, while other parameters remain the same as Cases 17 and 18.
- This multiplier results in a period equal to the modulus (100%).
- The plot shows that its 3-tuple diagram has no visible correlation.
- Conclusions: it is essential to examine PRNGs by performing numerous randomness tests, as the quality of a PRNG is highly dependent on using the “right” parameters $\{x_0, a, b, M\}$.

